National University of Computer and Emerging Sciences
Islamabad Operating System Lab Spring 2025

Lab Task Threads

Lab Task 1: Multi-Threaded Matrix Operations System

Scenario: A system needs to perform matrix computations in parallel to improve efficiency.

Thread Functions:

1. **Thread 1** → Computes the transpose of a given matrix.
2. **Thread 2** → Computes the determinant of the matrix.
3. **Thread 3** → Computes the inverse of the matrix (if possible).

Example Input (3×3 Matrix):

```
Matrix:

[ 1  2  3 ]

[ 4  5  6 ]

[ 7  8  9 ]
```

Expected Output:

```
Thread 1: Transpose computed:

[ 1  4  7 ]

[ 2  5  8 ]

[ 3  6  9 ]

Thread 2: Determinant computed: 0
```

Thread 3: Inverse cannot be computed (determinant is 0).

Matrix operations completed using multithreading!

Lab Task 2: Multi-Threaded Mathematical Computation System

Scenario: A system performs a multi-step numerical computation where each thread's output is needed by the next thread.

Thread Functions:

1. **Thread 1** → Generates the first N prime numbers.
2. **Thread 2** → Computes the sum of the generated prime numbers.
3. **Thread 3** → Finds the factorial of the sum computed by Thread 2.
4. **Parent** → Uses all results to compute the final expression:

$$\text{Final Result} = \frac{\text{Factorial}(\text{Sum of Primes})}{\text{Total Primes Found}}$$

Execution Flow:

Thread 1: Generates the first N prime numbers.
Thread 2: Waits for Thread 1, then computes the sum of those primes.
Thread 3: Waits for Thread 2, then finds the factorial of the sum.
(Parent): Waits for Thread 3, then computes the final result using all outputs.

Example Execution (N = 5):

Step-by-step output:

```
Thread 1: First 5 prime numbers: [2, 3, 5, 7, 11]

Thread 2: Sum of primes = 2 + 3 + 5 + 7 + 11 = 28

Thread 3: Factorial of 28 computed.

Thread 4: Final Result = Factorial(28) / 5 = (huge number) / 5

Computation completed using multithreading!
```

Lab Task 3: Multi-Threaded Ticket Booking System (Using `pthread_create()`, `pthread_self()`, and `pthread_exit()`)

Scenario: A movie ticket booking system is automated with **multiple threads**, each handling different customers independently.

Thread Functions:

1. **Thread 1 → Books a ticket for Customer 1 and prints its thread ID.**
2. **Thread 2 → Books a ticket for Customer 2 and exits.**
3. **Thread 3 → Books a ticket for Customer 3 and confirms their seat.**

Expected Output:

```
Thread 1: Booking ticket for Customer 1, Thread ID:
140439869986560
Thread 2: Booking ticket for Customer 2, exiting...
Thread 3: Booking ticket for Customer 3, Seat Confirmed!
All ticket bookings completed!
```

THE END
